

Sistema de ficheros en el espacio de usuario: FUSE

Grupo J: Javier Sánchez Alarcón y Francisco León Jaime

Introducción

En este trabajo se presenta miniEXT2, un sistema de ficheros implementado sobre FUSE (Filesystem in Userspace) que, aun siendo un subconjunto simplificado de EXT2, resulta suficientemente completo para ilustrar los mecanismos básicos de un sistema de archivos: gestión de bloques, inodos, directorios, enlaces y metadatos. A continuación, se describen brevemente las estructuras de datos utilizadas, las funciones auxiliares de bajo nivel y los manejadores de FUSE que implementan cada operación de usuario.

Estructuras de datos definidas en struct.h

Constantes y configuración general

Se han definido las siguientes constantes que determinan el tamaño del sistema de ficheros y cómo está dividido internamente:

- **FS_SIZE_BYTES**: Tamaño total del sistema de ficheros, ahora fijado en **8 MiB (8 * 1024 * 1024 bytes)**.
- **BLOCK_SIZE**: Tamaño de bloque individual, ahora ajustado a **512 bytes**, frente a los 4096 bytes anteriores.
- **NUM_BLOCKS**: Número total de bloques que resultan de dividir el tamaño del sistema entre el tamaño de cada bloque (en este caso, **16384 bloques**).
- **NUM_INODES**: Número total de inodos, establecido en **1024**.
- **INODE_SIZE**: Tamaño de cada inodo en bytes (**128 bytes**).
- **INODE_TABLE_BLK**: El bloque inicial donde se almacena la tabla de inodos, modificado al **bloque 5**.
- **FIRST_DATA_BLOCK**: Índice del primer bloque disponible para almacenar datos, mantenido en **bloque 10**.
- **MAX_NAME**: Tamaño máximo para el nombre de un archivo o directorio (**28 caracteres**).
- **DIRECT_POINTERS**: Número de punteros directos en cada inodo (**12 punteros**).
- **ROOT_INODE**: Índice del inodo que representa el directorio raíz (fijado en **0**).

Estructura *superblock*

La estructura *superblock* contiene metadatos generales del sistema de ficheros esenciales para su administración y uso:

- **total_blocks**: Cantidad total de bloques en el sistema.
- **total_inodes**: Número total de inodos.
- **block_size**: Tamaño de cada bloque en bytes.
- **free_blocks**: Cantidad de bloques actualmente disponibles.
- **free_inodes**: Número de inodos libres.
- **first_data_block**: Índice del primer bloque utilizable para almacenar datos.
- **fs_name**: Cadena que identifica al sistema de ficheros (limitada a **8 caracteres**).

Estructura *inode*

La estructura *inode* almacena metadatos específicos para cada archivo o directorio dentro del sistema de ficheros:

- **mode y uid**: Almacenan respectivamente el tipo de fichero (archivo, directorio) y los permisos, junto con el identificador del usuario propietario.
- **size**: Tamaño total del archivo o directorio.
- **blocks**: Número de bloques asignados al archivo o directorio.
- **direct[DIRECT_POINTERS]**: Array de **12 punteros directos** que apuntan a bloques concretos donde se almacenan los datos del fichero.
- **atime, ctime, mtime**: Marcas temporales (último acceso, última modificación de metadatos y última modificación de datos).
- **links_count**: Número de enlaces duros apuntando a este inodo.

Estructura *dir_entry*

Cada entrada de directorio (*dir_entry*) contiene:

- **inode**: Índice numérico del inodo al que apunta la entrada.
- **name[MAX_NAME]**: Nombre del archivo o subdirectorio correspondiente, limitado a 28 caracteres.

Variables globales

Por último, se definen las siguientes variables globales, empleadas en la implementación práctica:

- **fs_image:** Puntero que apunta al inicio del área mapeada en memoria que representa la imagen del sistema de ficheros completo.
- **sb:** Puntero que apunta directamente al superbloque.
- **inode_table:** Puntero que apunta directamente al área en memoria donde reside la tabla de inodos.

Estas modificaciones en las estructuras y constantes permiten gestionar eficazmente un sistema de ficheros compacto (8 MiB), adecuado para los objetivos didácticos del ejercicio, facilitando la comprensión profunda del manejo básico que hace un sistema operativo real de los ficheros y directorios, aunque simplificado y sin abarcar todos los aspectos avanzados presentes en un sistema real.

Funciones del sistema implementadas

Antes de detallar cada rutina, cabe destacar que **miniEXT2** cubre completamente los requisitos indicados en la práctica. A continuación, se listan las operaciones solicitadas junto con las funciones específicas del sistema FUSE que las implementan:

- **Creación de archivos:** Implementada por la función: `minifs_create(path, mode, fi)`.
- **Escritura y lectura del contenido de los archivos:** Implementada por las funciones:
 - Escritura: `minifs_write(path, buf, size, offset, fi)`
 - Lectura: `minifs_read(path, buf, size, offset, fi)`.
- **Borrado de archivos:** Implementada por la función: `minifs_unlink(path)`.
- **Renombrado de archivos (también usado para mover archivos entre directorios):** Implementada por la función: `minifs_rename(from, to)`.
- **Creación y borrado de directorios:** Implementada por las funciones:
 - Crear directorio: `minifs_mkdir(path, mode)`
 - Borrar directorio: `minifs_rmdir(path)`.
- **Renombrado de directorios (también para mover directorios):** Implementada también por la función: `minifs_rename(from, to)`.
- **Listado de directorios:** Implementada por la función: `minifs_readdir(path, buf, filler, off, fi)`.
- **Consulta del estado del sistema de ficheros (espacio libre, etc.) (opcional):** Implementada por la función: `minifs_statfs(path, stbuf)`.
- **Modificación/uso de permisos y propietarios (opcional):** Implementadas por las funciones:
 - Modificar permisos: `minifs_chmod(path, mode)`
 - Modificar propietarios: `minifs_chown(path, uid, gid)`.

- **Creación, borrado y consulta de enlaces duros y simbólicos (opcional):**

Implementadas por las funciones:

- Enlaces duros: `minifs_link(from, to)` y borrado indirecto a través de `minifs_unlink(path)`.
- Enlaces simbólicos: `minifs_symlink(target, linkpath)` y `minifs_readlink(path, buf, size)`.

Todas estas funciones están incluidas en la estructura de operaciones `fuse_operations` denominada `minifs_oper`, la cual permite al sistema FUSE reconocer y ejecutar correctamente las operaciones sobre el sistema de archivos:

```
static struct fuse_operations minifs_oper = {
    .getattr = minifs_getattr,
    .readdir = minifs_readdir,
    .open = minifs_open,
    .create = minifs_create,
    .truncate = minifs_truncate,
    .read = minifs_read,
    .write = minifs_write,
    .unlink = minifs_unlink,
    .mkdir = minifs_mkdir,
    .rmdir = minifs_rmdir,
    .rename = minifs_rename,
    .statfs = minifs_statfs,
    .chmod = minifs_chmod,
    .chown = minifs_chown,
    .link = minifs_link,
    .symlink = minifs_symlink,
    .readlink = minifs_readlink,
    .utimens = minifs_utimens,
    .access = minifs_access,
};
```

De este modo, el sistema **miniEXT2** satisface todas las operaciones requeridas, proporcionando una implementación completa y coherente de un sistema de ficheros FUSE simplificado para propósitos didácticos.

Manejadores de operaciones FUSE

Las operaciones definidas en la tabla `minifs_oper` cubren las llamadas más comunes de POSIX:

1. **getattr(path, &st):** Obtiene atributos de un fichero o directorio transfiriendo metadatos desde el inodo a una estructura `stat`.
2. **readdir(path, buf, filler, off, fi):** Lista contenidos de un directorio, rellenando entradas especiales (`.` y `..`) y activas.
3. **open(path, fi):** Verifica existencia y tipo de archivo, almacenando su inodo para futuras operaciones.
4. **create(path, mode, fi):** Crea nuevo fichero, asignando inodo y añadiendo su entrada en el directorio padre.
5. **truncate(path, size):** Permite truncar ficheros únicamente a tamaño cero, liberando bloques.
6. **write(path, buf, size, offset, fi):** Asigna bloques de ser necesario, copia datos y actualiza tamaño y tiempos.

7. **read(path, buf, size, offset, fi)**: Lee datos ajustando tamaño para evitar lecturas incorrectas y actualiza tiempo de acceso.
8. **unlink(path)**: Elimina ficheros, gestionando correctamente enlaces duros y recursos asociados.
9. **mkdir(path, mode)**: Crea directorios con entradas iniciales necesarias (. y ..).
10. **rmdir(path)**: Elimina directorios vacíos, asegurando integridad y consistencia del sistema.
11. **rename(from, to)**: Renombra o mueve archivos y directorios gestionando posibles conflictos.
12. **statfs(path, stbuf)**: Proporciona estadísticas del sistema (espacio disponible, tamaño bloque, etc.).
13. **chmod(path, mode)** y **chown(path, uid, gid)**: Cambian permisos y propietarios, actualizando metadatos relevantes.
14. **link(from, to)**: Crea enlaces duros, actualizando referencias a inodos existentes.
15. **symlink(target, linkpath)** y **readlink(path, buf, size)**: Manejan enlaces simbólicos, permitiendo almacenar y leer rutas.
16. **utimens(path, ts)**: Ajusta manualmente tiempos de acceso y modificación.
17. **access(path, mask)**: Verifica permisos sobre archivos o directorios según usuario actual.
Estas funciones aseguran una implementación completa y operativa del sistema miniEXT2.

Función principal (main)

1. **Mapeo de imagen**
Abre el archivo de imagen en modo lectura/escritura, obtiene su tamaño y lo mapea en memoria compartida (mmap).
2. **Inicialización de punteros**
Sitúa sb al inicio del superbloque y inode_table tras los tres bloques reservados.
3. **Reordenación de argumentos**
Ajusta el vector de parámetros para pasar primero las opciones de FUSE y, finalmente, el punto de montaje.
4. **Invocación de FUSE**
Lanza fuse_main con la tabla de operaciones minifs_oper, quedando el demonio a la espera de llamadas al sistema de ficheros.

A continuación, desarrollamos de manera más detallada cada una de las funciones auxiliares y manejadores de FUSE de **miniEXT2**, explicando su propósito, funcionamiento interno y consideraciones de diseño.

Funciones auxiliares de acceso y manipulación de estructuras

Antes de profundizar en cada llamada FUSE que satisface los requisitos de la práctica, cabe destacar que **miniEXT2** se apoya en una serie de rutinas auxiliares de bajo nivel que facilitan el acceso y la manipulación de la imagen de disco. Estas funciones auxiliares son:

I. get_inode(i) y get_block(b)

En **miniEXT2** resulta esencial abstraer el cálculo de punteros sobre la imagen mapeada. La función

```
static struct inode *get_inode(uint32_t i) {
    return &inode_table[i];
}
```

devuelve directamente la dirección del inodo número *i* dentro del array inode_table, que definimos como

```
inode_table = (struct inode *) (fs_image + INODE_TABLE_BLK *
                                BLOCK_SIZE);
```

De este modo, acceder a cualquier campo de un inodo —por ejemplo get_inode(idx)->mode o get_inode(idx)->size— se hace tan sencillo como desreferenciar un puntero. Por su parte,

```
static void *get_block(uint32_t b) {
    return fs_image + b * sb->block_size;
}
```

calcula la dirección del bloque de datos *b* multiplicando su índice por el tamaño (sb->block_size, habitualmente 4096 B) y sumándolo al puntero base fs_image. Así, operaciones de lectura o escritura en disco se traducen en simples

```
memcpy(get_block(b) + offset, buf, len).
```

II. split_path(path, dirpath, basename)

Para crear o eliminar un elemento, necesitamos separar la ruta en carpeta padre y nombre de archivo. Observa este fragmento:

```
const char *s = strrchr(path, '/');
if (!s || s == path) {
    strcpy(dirpath, "");
    strcpy(basename, s ? s+1 : path);
} else {
    size_t len = s - path;
    strncpy(dirpath, path, len);
    dirpath[len] = '\0';
    strcpy(basename, s+1);
}
```

Aquí, strrchr busca la última '/'. Cuando la encuentra en posición intermedia, copia exactamente los primeros len caracteres en dirpath y ajusta el terminador nulo, mientras que basename recibe todo lo que sigue a la barra. Esto garantiza que rutas como "/foo/bar.txt" produzcan dirpath="/foo" y basename="bar.txt", facilitando la inserción o borrado en el directorio correcto.

III. resolve_path(path)

La función de resolución de rutas recorre jerárquicamente cada componente. Fíjate en este bucle clave:

```
char tmp[PATH_MAX];
strcpy(tmp, path+1, sizeof(tmp)-1);
tok = strtok(tmp, "/");
int curr = ROOT_INODE;
```

```

while (tok) {
    struct inode *d = get_inode(curr);
    if (!(d->mode & S_IFDIR)) return -ENOENT;

    struct dir_entry *e = get_block(d->direct[0]);
    int n = d->size / sizeof(*e);
    bool found = false;

    for (int i = 0; i < n; i++) {
        if (e[i].inode && strcmp(e[i].name, tok) == 0) {
            curr = e[i].inode;
            found = true;
            break;
        }
    }
    if (!found) return -ENOENT;
    tok = strtok(NULL, "/");
}
return curr;

```

Tras copiar la ruta sin la barra inicial, strtok separa cada componente. Para cada tok, comprobamos primero que el inodo actual represente un directorio (d->mode & S_IFDIR). Luego cargamos su bloque de entradas (direct[0]) y lo recorremos entero —calculando $n = d->size / \text{sizeof}(*e)$ — buscando una coincidencia en e[i].name. Si la hallamos, actualizamos curr; si no, devolvemos -ENOENT. Finalmente, curr contiene el inodo apuntado por la ruta completa.

IV. add_entry(pd, name, child)

La inserción de un nuevo fichero o subdirectorio en un directorio padre pd se apoya en este fragmento:

```

struct dir_entry *e = get_block(d->direct[0]);
int max = sb->block_size / sizeof(*e);
for (int i = 0; i < max; i++) {
    if (!e[i].inode) {
        e[i].inode = child;
        strncpy(e[i].name, name, MAX_NAME-1);
        e[i].name[MAX_NAME-1] = '\0';
        d->size += sizeof(*e);
        d->mtime = time(NULL);
        msync(e, sb->block_size, MS_SYNC);
        return 0;
    }
}
return -ENOSPC;

```

Tras obtener el bloque de directorio, calculamos cuántas entradas caben (max). El bucle busca la primera ranura libre (inode == 0), asigna allí el número de inodo child, copia el nombre con seguridad contra desbordamiento y ajusta el tamaño y la fecha de modificación del directorio. Finalmente, msync garantiza que los cambios queden grabados en la imagen antes de continuar.

V. remove_entry(pd, name)

Para borrar, seguimos una lógica simétrica:

```

struct dir_entry *e = get_block(d->direct[0]);
int max = sb->block_size / sizeof(*e);
for (int i = 0; i < max; i++) {
    if (e[i].inode && strcmp(e[i].name, name) == 0) {
        e[i].inode = 0;
        e[i].name[0] = '\0';
        d->mtime = time(NULL);
        msync(e, sb->block_size, MS_SYNC);
        return 0;
    }
}

```

```

    }
}
return -ENOENT;

```

Aquí, cuando encontramos el nombre exacto, limpiamos la ranura (inode = 0, name[0]='\0'), actualizamos el tiempo y sincronizamos. Si no lo hallamos, devolvemos -ENOENT, permitiendo a la función superior decidir si la ruta no existía.

VI. alloc_block()

Esta función busca el primer bloque de datos libre. La novedad en esta versión es que **no se usa un mapa de bits**, sino que se comprueba directamente si algún inodo ya lo está utilizando:

```

for (uint32_t b = sb->first_data_block; b < sb->total_blocks; b++) {
    bool usado = false;

    // Recorremos todos los inodos
    for (uint32_t i = 0; i < sb->total_inodes; i++) {
        struct inode *in = get_inode(i);

        // Comprobamos si alguno de los punteros directos apunta a b
        for (int j = 0; j < DIRECT_POINTERS; j++) {
            if (in->direct[j] == b) {
                usado = true;
                break;
            }
        }
        if (usado) break;
    }

    if (!usado) {
        sb->free_blocks--;
        msync(sb, sizeof(*sb), MS_SYNC);
        return b;
    }
}
return -ENOSPC; // No queda espacio libre

```

La función itera desde first_data_block hasta total_blocks, y para cada bloque, examina si algún inodo lo tiene referenciado en sus punteros directos. Si encuentra uno libre, lo marca como ocupado decrementando free_blocks y devuelve su número.

VII. alloc_inode()

Busca el primer inodo libre, comprobando su campo mode. Si está a cero, se considera disponible:

```

for (uint32_t i = 1; i < sb->total_inodes; i++) {
    if (!get_inode(i)->mode) {
        sb->free_inodes--;
        msync(sb, sizeof(*sb), MS_SYNC);
        return i;
    }
}
return -ENOSPC;

```

Se omite el índice 0 (reservado para el directorio raíz). Una vez localizado un inodo libre, se reduce el contador free_inodes y se sincroniza con disco usando msync.

VIII. minifs_getattr(path, st)

Esta función implementa la operación getattr() de FUSE, que se encarga de **rellenar la estructura stat** con los metadatos de un archivo o directorio, como permisos, tamaño, tiempos, propietario, etc.:

```
int idx = resolve_path(path);
if (idx < 0) return idx;
struct inode *n = get_inode(idx);
memset(st, 0, sizeof(*st));
st->st_mode = (n->mode & S_IFMT) | (n->mode & 0777);
st->st_nlink = n->links_count;
st->st_size = n->size;
st->st_atime = n->atime;
st->st_mtime = n->mtime;
st->st_ctime = n->ctime;
st->st_uid = n->uid;
st->st_gid = n->gid;
```

1.- Resolución de ruta: La función primero convierte la ruta a su inodo correspondiente con `resolve_path(path)`. Si no se encuentra, devuelve un error.

2.- Inicialización: Se limpia la estructura `stat` usando `memset()`.

3.- Asignación de campos:

- `st_mode`: Se separan los bits de tipo de archivo (`S_IFMT`) y los permisos (`0777`) contenidos en el campo `mode` del inodo.
- `st_nlink`, `st_size`, `st_uid`, `st_gid`: Se asignan directamente desde los valores del inodo.
- `st_atime`, `st_mtime`, `st_ctime`: Representan los tiempos de último acceso, modificación del contenido y modificación de metadatos, respectivamente.

Gracias a esta función, comandos como `ls -l`, `stat` o cualquier programa que consulte atributos POSIX puede obtener la información adecuada desde el sistema miniEXT2.

IX. `minifs_readdir(path, buf, filler, off, fi)`

El listado de directorio sigue esta pauta:

```
static int minifs_readdir(const char *path, void *buf,
                        fuse_fill_dir_t filler,
                        off_t off, struct fuse_file_info *fi) {
    (void)off; (void)fi;
    int idx = resolve_path(path);
    if (idx < 0) return idx;
    struct inode *d = get_inode(idx);
    if (!(d->mode & S_IFDIR)) return -ENOTDIR;

    filler(buf, ".", NULL, 0);
    filler(buf, "..", NULL, 0);

    struct dir_entry *e = get_block(d->direct[0]);
    int n = d->size / sizeof(struct dir_entry);
    for (int i = 0; i < n; i++) {
        if (e[i].inode &&
            strcmp(e[i].name, ".") &&
            strcmp(e[i].name, "..")) {
            filler(buf, e[i].name, NULL, 0);
        }
    }
    return 0;
}
```

Tras verificar que `d->mode & S_IFDIR`, rellenamos primero los directorios especiales y luego todas las entradas activas—omitiendo de nuevo “.” y “..”.

X. `minifs_open(path, fi)`

Para abrir, basta con:

```
int idx = resolve_path(path);
if (idx < 0) return -ENOENT;
if (get_inode(idx)->mode & S_IFDIR) return -EISDIR;
fi->fh = idx;
```

Así las operaciones posteriores de lectura y escritura utilizan `fi->fh` como referencia directa al inodo.

XI. `minifs_create(path, mode, fi)`

Crear un fichero integra varios pasos clave:

```
static int minifs_create(const char *path, mode_t mode,
                        struct fuse_file_info *fi) {
    char dp[PATH_MAX], bn[MAX_NAME];
    split_path(path, dp, bn);
    int pd = resolve_path(dp);
    if (pd < 0) return pd;
    struct inode *d = get_inode(pd);
    if (!(d->mode & S_IFDIR)) return -ENOTDIR;
    int in = alloc_inode();
    if (in < 0) return in;
    struct inode *n = get_inode(in);
    memset(n, 0, sizeof(*n));
    n->mode = S_IFREG | (mode & 0777);
    n->uid = getuid();
    n->gid = getgid();
    n->size = 0;
    n->blocks = 0;
    n->ctime = n->mtime = n->atime = time(NULL);
    n->links_count = 1;
    int r = add_entry(pd, bn, in);
    if (r < 0) return r;
    fi->fh = in;
    return 0;
}
```

Inicializamos el inodo, asignamos permisos y propietario, fijamos tiempos y contador de enlaces, y enlazamos el nuevo inodo en su directorio padre.

XII. `minifs_truncate(path, size)`

La función `truncate()` de FUSE se utiliza para **cambiar el tamaño de un archivo**. En miniEXT2, esta operación solo está implementada para vaciar archivos (es decir, `size == 0`):

```
int idx = resolve_path(path);
if (idx < 0) return idx;
struct inode *n = get_inode(idx);

if (n->mode & S_IFDIR) return -EISDIR;
if (!(n->mode & S_IWUSR)) return -EACCES;
if (size != 0) return -EINVAL;

// Liberar todos los bloques directos usados
for (int i = 0; i < DIRECT_POINTERS; i++) {
    if (n->direct[i]) {
        memset(get_block(n->direct[i]), 0, sb->block_size);
        sb->free_blocks++;
        n->direct[i] = 0;
    }
}
n->blocks = 0;
n->size = 0;
n->mtime = time(NULL);
```

Se resuelve la ruta con `resolve_path(path)`. Si el inodo apunta a un directorio (`S_IFDIR`), se devuelve error `-EISDIR`. Si el usuario no tiene permisos de escritura (`S_IWUSR`), se devuelve `-EACCES`. Actualmente, solo se permite truncar a tamaño **cero**. Cualquier otro valor de `size` da error `-EINVAL`.

Con respecto a la liberación de bloques, se recorren los punteros directos del inodo. Si apuntan a bloques asignados, se limpian (con `memset`), se marcan como libres incrementando `free_blocks`, y se anulan los punteros.

Con respecto a la actualización de metadatos, se pone el tamaño del archivo a 0. Se pone a 0 el número de bloques usados (blocks). Se actualiza el `mtime` con la hora actual.

Con esta implementación, comandos como `truncate -s 0` o sobrescripciones que reduzcan el tamaño del archivo se comportan correctamente, liberando recursos y actualizando los metadatos.

XIII. `minifs_write(path, buf, size, offset, fi)`

La función `minifs_write` es la encargada de gestionar la escritura de datos en un archivo del sistema de ficheros `miniEXT2`. Se parte del inodo correspondiente al descriptor de archivo abierto, que se obtiene a través de `get_inode(fi->fh)`. El sistema verifica que dicho inodo represente efectivamente un archivo regular y que el usuario tenga permisos de escritura. Si no se cumple alguna de estas condiciones, la función devuelve un error, como `-EISDIR` o `-EACCES`:

```
static int minifs_write(const char *path, const char *buf,
                      size_t size, off_t offset,
                      struct fuse_file_info *fi) {
    (void)path;
    struct inode *n = get_inode(fi->fh);
    if (!(n->mode & S_IFREG)) return -EISDIR;
    if (!(n->mode & S_IWUSR)) return -EACCES;
```

El proceso de escritura se realiza bloque a bloque. Para ello, se calcula el índice del bloque lógico (`blk_idx`) y el desplazamiento dentro del bloque (`blk_off`) a partir del `offset` inicial y de los bytes ya escritos. Se determina cuántos bytes se pueden escribir en ese bloque (`to_write`) y se ajusta si es necesario para no sobrepasar los límites del búfer.

Si el bloque aún no está asignado, se solicita uno nuevo con `alloc_block()`. En caso de éxito, se asigna al campo correspondiente en `n->direct[]` y se actualiza el contador de bloques usados (`n->blocks`). Luego, los datos se copian al bloque real mediante `memcpy`.

```
size_t written = 0;
while (written < size) {
    int blk_idx = (offset + written) / sb->block_size;
    int blk_off = (offset + written) % sb->block_size;
    size_t to_write = sb->block_size - blk_off;
    if (to_write > size - written) to_write = size - written;
    if (blk_idx < DIRECT_POINTERS) {
        if (!n->direct[blk_idx]) {
            int b = alloc_block();
            if (b < 0) return written ? (int)written : b;
            n->direct[blk_idx] = b;
            n->blocks++;
        }
    }
```

```
memcpy(get_block(n->direct[blk_idx]) + blk_off, buf + written,
       to_write);
}
written += to_write;
```

Al finalizar la escritura, si esta ha ampliado el archivo, se actualiza el tamaño (`n->size`) con el mayor entre el valor actual y `offset + written`. También se registra el momento de modificación (`n->mtime`) con `time(NULL)`. Finalmente, la función devuelve el número total de bytes escritos.

```
n->size = offset + written > n->size ? offset + written : n->size;
n->mtime = time(NULL);
return written;
```

A pesar de su sencillez, esta implementación permite comprender cómo un sistema de archivos gestiona el almacenamiento físico de datos sobre bloques, cómo reserva espacio y cómo actualiza la estructura lógica que lo representa.

XIV. `minifs_read(path, buf, size, offset, fi)`

La función `minifs_read` permite leer el contenido de un archivo almacenado en el sistema de ficheros. Comienza obteniendo el inodo asociado al descriptor de archivo abierto (`fi->fh`) mediante `get_inode`. Una vez recuperado el inodo, se verifica que este represente un fichero regular (`S_IFREG`) y que el usuario tenga permisos de lectura. Si se trata de un directorio o el acceso está denegado, se devuelve el error correspondiente.

```
static int minifs_read(const char *path, char *buf, size_t size,
                      off_t offset, struct fuse_file_info *fi) {
    (void)path;
```

```
    struct inode *n = get_inode(fi->fh);
    if (!(n->mode & S_IFREG)) return -EISDIR;
    if (!(n->mode & S_IRUSR)) return -EACCES;
```

La operación de lectura se realiza bloque a bloque. Se calcula el índice del bloque (`blk_idx`) y el desplazamiento dentro del mismo (`blk_off`) en función del `offset` y la cantidad de bytes ya leídos (`read`). A partir de ahí se determina el tamaño máximo que puede leerse en el bloque actual (`to_read`), asegurándose de no superar ni el tamaño del bloque ni el tamaño total del archivo.

Si el bloque directo correspondiente (`n->direct[blk_idx]`) está asignado, se copia el contenido desde la imagen del sistema de archivos al búfer del usuario utilizando `memcpy`.

```
size_t read = 0;
while (read < size && offset + read < n->size) {
    int blk_idx = (offset + read) / sb->block_size;
    int blk_off = (offset + read) % sb->block_size;
    size_t to_read = sb->block_size - blk_off;
    if (to_read > size - read) to_read = size - read;
    if (to_read > n->size - (offset + read)) to_read = n->size - (offset + read);

    if (blk_idx < DIRECT_POINTERS && n->direct[blk_idx]) {
        memcpy(buf + read, get_block(n->direct[blk_idx]) + blk_off,
               to_read);
    }
    read += to_read;
}
```

Finalmente, se actualiza el tiempo de acceso del inodo (`n->atime`) para reflejar que el archivo ha sido leído, y se retorna el número de bytes leídos.

```
n->atime = time(NULL);
return read;
```

Esta función es un buen ejemplo de cómo, a nivel de sistema de ficheros, la lectura no es más que una operación de localización de bloques físicos y copia de su contenido. También muestra la necesidad de tener en cuenta los límites de tamaño del archivo y los permisos de acceso en cada operación.

XV. minifs_unlink(path)

La función `minifs_unlink` implementa la eliminación de archivos ordinarios del sistema de ficheros. El primer paso consiste en dividir la ruta completa del archivo en su directorio padre y su nombre base mediante `split_path`. Posteriormente, se resuelven ambas rutas con `resolve_path` para obtener los índices de inodo tanto del directorio padre como del archivo objetivo.:

```
static int minifs_unlink(const char *path) {
    char dp[PATH_MAX], bn[MAX_NAME];
    split_path(path, dp, bn);
    int pd = resolve_path(dp);
    if (pd < 0) return pd;
    int idx = resolve_path(path);
    if (idx < 0) return idx;
```

Una vez que se tiene el inodo del archivo a eliminar, se comprueba si se trata de un directorio. Si lo es, se devuelve `-EISDIR`, ya que `unlink` está destinado únicamente a archivos regulares. A continuación, se gestiona el caso de enlaces duros. Si el contador de enlaces (`links_count`) es mayor que uno, simplemente se decrementa y se elimina la entrada del directorio padre con `remove_entry`, sin liberar recursos.

```
struct inode *n = get_inode(idx);
if (n->mode & S_IFDIR) return -EISDIR;
if (n->links_count > 1) {
    n->links_count--;
    return remove_entry(pd, bn);
}
```

Si el archivo tiene solo un enlace (es decir, se va a eliminar por completo), se procede a liberar los recursos asociados. Si el inodo tiene bloques asignados, se borra el contenido del primer bloque directo con `memset`, se incrementa el contador de bloques libres en el superbloque, y se limpian las referencias en el inodo. Después, se borra el inodo entero con otro `memset`, y se incrementa el contador de inodos libres.

```
if (n->blocks) {
    memset(get_block(n->direct[0]), 0, sb->block_size);
    sb->free_blocks++;
    n->blocks = 0;
    n->direct[0] = 0;
}
memset(n, 0, sizeof(*n));
sb->free_inodes++;
```

Finalmente, se elimina la entrada correspondiente del directorio padre mediante `remove_entry`. Así, se asegura que el archivo queda completamente eliminado del sistema de ficheros.

```
return remove_entry(pd, bn);
```

Esta función muestra claramente cómo un sistema de ficheros gestiona la eliminación lógica de un archivo, diferenciando entre un simple enlace duro que se elimina y la verdadera liberación de los recursos físicos cuando se elimina el último enlace.

XVI. minifs_mkdir(path, mode) y minifs_rmdir(path)

La función `minifs_mkdir` se encarga de crear un nuevo directorio en el sistema de ficheros. Como paso inicial, se divide la ruta especificada en la ruta del directorio padre (`dp`) y el nombre del nuevo directorio (`bn`) usando `split_path`. Luego, se resuelve el índice de inodo del directorio padre mediante `resolve_path`. Si este no existe o no es realmente un directorio, se devuelve un error apropiado.

```
char dp[PATH_MAX], bn[MAX_NAME];
split_path(path, dp, bn);
int pd = resolve_path(dp);
if (pd < 0) return pd;
struct inode *parent = get_inode(pd);
if (!(parent->mode & S_IFDIR)) return -ENOTDIR;
```

Una vez validado el directorio padre, se solicita un nuevo inodo y un bloque de datos mediante `alloc_inode` y `alloc_block`. Si alguno de estos recursos no está disponible, se devuelve el error correspondiente.

```
int in = alloc_inode();
if (in < 0) return in;
int b = alloc_block();
if (b < 0) return b;
```

El nuevo inodo se inicializa con los valores adecuados: se configura su modo como directorio (con los permisos indicados), se asignan los identificadores de usuario y grupo actuales, y se actualizan los tamaños, número de bloques, y tiempos de acceso, modificación y creación. Además, se establece `links_count` en 2, ya que un directorio siempre tiene al menos dos enlaces: "." (a sí mismo) y ".." (al padre).

```
struct inode *n = get_inode(in);
memset(n, 0, sizeof(*n));
n->mode = S_IFDIR | (mode & 0777);
n->uid = getuid();
n->gid = getgid();
n->size = 2 * sizeof(struct dir_entry);
n->blocks = 1;
n->direct[0] = b;
n->atime = n->ctime = n->mtime = time(NULL);
n->links_count = 2;
```

Se limpia el contenido del bloque asignado y se insertan las dos entradas obligatorias: "." apuntando al nuevo inodo, y ".." apuntando al inodo del directorio padre.

```
void *blk = get_block(b);
memset(blk, 0, sb->block_size);

struct dir_entry *e = blk;
e[0].inode = in; strcpy(e[0].name, ".");
e[1].inode = pd; strcpy(e[1].name, "..");
```

Como último paso, se incrementa el número de enlaces del directorio padre (por el nuevo subdirectorio creado) y se registra la nueva entrada en dicho directorio mediante `add_entry`.

```
parent->links_count++;
return add_entry(pd, bn, in);
```

Este procedimiento asegura que el nuevo directorio queda correctamente enlazado dentro de la jerarquía del sistema de ficheros y es coherente con la estructura de directorios de EXT2.

XVII. minifs_rename(from, to)

Mover o renombrar un fichero o directorio implica trabajar con dos rutas: la de origen y la de destino. En primer lugar, dividimos ambas rutas en directorio padre y nombre mediante `split_path`, y resolvemos sus inodos con `resolve_path`. Si en la ruta destino ya existe algo, comprobamos si es un directorio no vacío; en ese caso devolvemos `-ENOTEMPTY`. El fragmento central es:

```
// Si ya existe destino, lo borramos
int ti = resolve_path(to);
if (ti >= 0) {
    struct inode *nd = get_inode(ti);
    if ((nd->mode & S_IFMT) == S_IFDIR && nd->size > 2*sizeof(struct
dir_entry))
        return -ENOTEMPTY;
    remove_entry(tpd, tn);
    memset(nd, 0, sizeof(*nd));
    sb->free_inodes++;
}
// Añadimos la nueva entrada y eliminamos la antigua
add_entry(tpd, tn, fi);
remove_entry(fpd, fn);
```

Con ello, el inodo original se mueve dentro del mismo sistema sin copiar datos, respetando las reglas POSIX sobre directorios y reemplazos.

XVIII. minifs_statfs(path, stbuf)

Para que utilidades como `df` puedan obtener estadísticas de espacio, `statfs` copia directamente los valores del superbloque a la estructura `statvfs`:

```
stbuf->f_bsize = sb->block_size;
stbuf->f_blocks = sb->total_blocks;
stbuf->f_bfree = sb->free_blocks;
stbuf->f_files = sb->total_inodes;
stbuf->f_ffree = sb->free_inodes;
stbuf->f_namemax = MAX_NAME;
```

De esta manera, se expone el número total y libre de bloques e inodos y la longitud máxima de nombre de fichero, facilitando la consulta de la capacidad del sistema.

XIX. minifs_chmod(path, mode)

Cambiar permisos es tan sencillo como actualizar los bits de modo manteniendo el tipo de objeto e introducir una nueva marca de tiempo de cambio (`ctime`):

```
struct inode *n = get_inode(idx);
n->mode = (n->mode & S_IFMT) | (mode & 0777);
n->ctime = time(NULL);
```

Con ello, `chmod` sobre la ruta reflejará inmediatamente los nuevos permisos cuando se consulte con `ls -l` o similares.

XX. minifs_chown(path, uid, gid)

Modificar el propietario de un fichero se realiza comprobando si el uid pedido no es `-1` y asignándolo, además de actualizar la fecha de modificación (`mtime`):

```
int idx = resolve_path(path);
if (idx < 0) return idx;
struct inode *n = get_inode(idx);
if (uid != (uid_t)-1) n->uid = uid;
if (gid != (gid_t)-1) n->gid = gid;
n->mtime = time(NULL);
return 0;
```

Aunque `gid` no se usa en esta versión, podríamos extender el inodo para almacenarlo de manera similar.

XXI. minifs_link(from, to)

Crear un hard link suma un nuevo nombre al mismo inodo, incrementando su contador de enlaces:

```
int fi = resolve_path(from);
struct inode *src = get_inode(fi);
add_entry(pd, bn, fi);
src->links_count++;
src->mtime = time(NULL);
```

Esto hace que ambos nombres apunten a la misma estructura de datos en disco, liberándose realmente cuando el contador llegue a cero.

XXII. minifs_symlink(target, linkpath)

Los enlaces simbólicos requieren almacenar la ruta destino como contenido de fichero:

```
static int minifs_symlink(const char *target, const char *linkpath) {
    char dp[PATH_MAX], bn[MAX_NAME];
    split_path(linkpath, dp, bn);
    int pd = resolve_path(dp);
    if (pd < 0) return pd;
    int in = alloc_inode();
    if (in < 0) return in;
    int b = alloc_block();
    if (b < 0) return b;
    struct inode *n = get_inode(in);
    memset(n, 0, sizeof(*n));
    n->mode = S_IFLNK | 0777;
    n->uid = getuid();
    n->gid = getgid();
    n->size = strlen(target);
    n->blocks = 1;
    n->direct[0] = b;
    n->ctime = n->mtime = n->atime = time(NULL);
    n->links_count = 1;
    memcpy(get_block(b), target, n->size);
    return add_entry(pd, bn, in);
}
```


Así, el bloque asignado guarda la cadena target, y el inodo marcado con S_IFLNK permite diferenciarlo de un fichero normal.

XXIII. minifs_readlink(path, buf, size)

Para leer un symlink, basta con copiar su contenido al búfer del usuario, respetando el límite size:

```
struct inode *n = get_inode(idx);
size_t len = n->size < size-1 ? n->size : size-1;
memcpy(buf, get_block(n->direct[0]), len);
buf[len] = '\0';
```

De este modo, readlink devuelve la ruta original que se almacenó al crear el enlace.

XXIV. minifs_utimens(path, ts)

Ajustar marcas de tiempo de acceso y modificación usa directamente el array de struct timespec:

```
struct inode *n = get_inode(idx);
n->atime = ts[0].tv_sec;
n->mtime = ts[1].tv_sec;
```

Esta función permite replicar comportamientos avanzados de POSIX, como preservar tiempos al copiar ficheros.

XXV. minifs_access(path, mask)

Antes de permitir operaciones sobre un fichero, comprobamos los permisos del usuario frente a los bits solicitados:

```
mode_t m = n->mode & 0777;
if ((mask & R_OK) && !(m & S_IRUSR)) return -EACCES;
if ((mask & W_OK) && !(m & S_IWUSR)) return -EACCES;
if ((mask & X_OK) && !(m & S_IXUSR)) return -EACCES;
```

Así, cualquier intento de lectura, escritura o ejecución se valida coordinadamente con los permisos almacenados en el inodo.

Programa formateador de la imagen FUSE

Además del sistema de ficheros propiamente dicho, **miniEXT2** incluye un pequeño programa en C cuya finalidad es “formatear” la imagen antes de ponerla en marcha. Este formateador se encarga de inicializar el superbloque, la tabla de inodos y el bloque de datos correspondiente al directorio raíz.

Para empezar, el formateador recibe como parámetro el nombre de un fichero de imagen (por ejemplo, imagen) y un número total de inodos deseado. Al abrir ese fichero en modo lectura/escritura, calcula cuántos bloques de tamaño BLOCK_SIZE bytes caben en su tamaño total. Esa división entre el tamaño del fichero y BLOCK_SIZE define total_blocks, la capacidad del sistema de ficheros en forma de bloques.

A continuación, el formateador escribe un **superbloque** en el primer bloque (bloque 0). Ese superbloque contiene todos los datos globales del sistema: número total de bloques, número total de inodos, tamaños de bloque e inodo, contadores de

bloques libres e inodos libres, índice del primer bloque de datos y un nombre identificativo (“MiniEXT2”). Con esta información, cualquier llamada posterior de FUSE sabrá cuántos bloques existen, cuántos quedan libres y dónde empieza el área de datos dentro de la imagen.

Después de volcar el superbloque, el programa posiciona el cursor en el bloque 3 (índice INODE_TABLE_BLK = 3) y construye allí la **tabla de inodos**. Cada inodo ocupa INODE_SIZE bytes, de modo que al escribir total_inodes estructuras de tamaño fijo se consumen tantos bloques consecutivos como sean necesarios (desde el bloque 3 en adelante).

En esta fase, el formateador inicializa el **inodo 0** como el directorio raíz: le asigna permiso de directorio (S_IFDIR) con rwxr-xr-x (0755), usuario 0, tamaño suficiente para dos entradas (“.” y “..”), un único bloque de datos (apuntando luego al bloque 10) y un contador de enlaces en dos (uno por “.” y otro por “..”). Todos los inodos restantes (del 1 al total_inodes-1) se dejan a cero, lo que significa que están disponibles para ser asignados más adelante por las rutinas de FUSE (alloc_inode).

Aunque el espacio para el directorio raíz se reserva correctamente, en esta versión las entradas “.” y “..” están comentadas, por lo que su creación real podría diferirse hasta el momento de montaje o ser gestionada por alguna otra rutina.

La llamada a fwrite copia esos 4096 bytes en la imagen, de modo que el directorio raíz ya arranca con sus referencias “.” y “..” apuntando al inodo 0. El resto del bloque se deja a ceros, y todos los bloques siguientes (11, 12, ...) permanecen igualmente llenos de ceros, indicando que aún están libres para almacenar archivos o subdirectorios conforme el usuario realice operaciones FUSE.

De esta manera, el formateador transforma un fichero “plano” en un sistema de ficheros completo y consistente:

1. **Superbloque** (bloque 0) con la información global.
2. **Tabla de inodos** (bloques 3 a ...) en la que el inodo 0 se usa como raíz y los demás quedan libres.
3. **Bloque de datos del directorio raíz** (bloque 10) con entradas “.” y “..”.
4. **Resto de bloques e inodos a cero**, listos para asignar dinámicamente durante la ejecución de las llamadas POSIX en FUSE.

A partir de ese momento, al montar la imagen con el programa FUSE, cada petición de lectura o escritura de bloque se traduce en un desplazamiento dentro de este único fichero de imagen. De este modo, todas las estructuras de metadatos (superbloque, inodos, tablas de directorio) y el contenido real de archivos y subdirectorios residen íntegramente dentro de imagen.fs, exactamente como si se tratara de un dispositivo de bloques real en un sistema UNIX clásico.

Mejoras

Se ha realizado un sistema bastante simplificado por lo que hay espacio para muchas mejoras y modificaciones, pero tal y como se encuentra el proyecto éstas son las que vemos más interesantes y prioritarias:

- Uso de bitmaps de bloques e inodos.
- Uso de punteros indirectos para trabajar con mayor tamaño de ficheros.
- Mejoras generales del código para simplificar y tener el código más ordenado.

Conclusiones

En general, este proyecto nos ha servido para aprender bastante sobre funcionamiento interno de un sistema de ficheros.

Ha sido un gran reto comprender las estructuras necesarias para su funcionamiento y como se relacionan entre sí para después implementar sobre ellas las funciones del sistema FUSE que es realmente complicado trabajar con él.

Aún así y con todos los problemas con los que nos hemos ido encontrando hemos sacado un proyecto funcional que cumple con el objetivo didáctico de la asignatura.

Bibliografía

- [libfuse: fuse_operations Struct Reference](#)
- [libfuse/libfuse: The reference implementation of the Linux FUSE \(Filesystem in Userspace\) interface](#)
- [francmUMA/DSOA2023_inode_fs](#)
- [ext2 - Wikipedia](#)
- [Anatomy of the Linux virtual file system switch – IBM Developer](#)
- [13. The Virtual Filesystem | Linux Kernel Development, Third Edition](#)